

```

# Basic Plot - Scatter Plot
sns.scatterplot(data=iris, x='sepal_length', y='sepal_width', hue='species')
# count plot on two categorical variable
sns.countplot(x='sex', hue='smoker', data=tips)
# count plot on single categorical variable
sns.countplot(x='sex', data=tips)
# Categorical Plot
sns.catplot(data=tips, x='day', y='total_bill', kind='swarm', hue='sex')
# Pair Plot
sns.pairplot(iris, hue='species')
# Violin Plot
sns.violinplot(data=tips, x='day', y='total_bill', hue='sex', split=True)
# Heatmap
corr = tips.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm')
#boxplot
sns.boxplot(data=tips, x="day", y="total_bill", hue="sex", notch=True, palette="Set2")
# Histogram and KDE Plot
sns.histplot(data=tips, x='total_bill', kde=True)
# Line Plot
sns.lineplot(data=flights, x='year', y='passengers')

```

```

x = np.array([1,2,3,4,5])
y = np.array([5,2,3,1,4])
plt.figure(figsize=(4,4))
plt.title("DATA",fontsize=30)
plt.plot(x,y,color='red',marker='.',linestyle='--')
plt.scatter(x,y,color='blue',marker='o')
plt.barh(categories,values,color='orange')
plt.hist(data,bins=30)
plt.xlabel("X-AXIS",fontsize=20)
plt.ylabel("Y-AXIS",fontsize=20)
plt.yticks(fontsize=15)
plt.xticks(fontsize=15)
plt.ylim(0,10)
plt.xlim(0,10)
plt.show()
plt.subplot(2,2,1)

```

```

# Step 1: Create and write to a file
with open("example.txt", "w") as file:
    file.write("Hello, this is a file handling example in Python.\n")
    file.write("We are writing multiple lines to this file.\n")

# Step 2: Read the file
with open("example.txt", "r") as file:
    content = file.read()
    print("File Content:\n", content)

```

```

arr3 = np.array([1, 2, 3], dtype='float64')
zeros_arr = np.zeros((3, 4))
ones_arr = np.ones((2, 3))
range_arr = np.arange(0, 10, 2) # Start, stop, step
linspace_arr = np.linspace(0, 1, 5) # Start, stop, number of elements
random_arr = np.random.rand(2, 2)
arr2.shape, arr3.dtype, arr2.ndim, arr2.size, arr1.reshape(5, 1), arr2[0:2, 1:3] #indexing
np.sum(arr1), np.mean(arr1), np.std(arr1), np.max(arr1), np.min(arr1)
np.dot(arr_a, arr_b), np.matmul(matrix_a, matrix_b)
matrix_a.T # transpose
np.loadtxt("my_array.txt", delimiter=",")

```

```

df.duplicated() # Check for duplicates
df.drop_duplicates(inplace=True) # Remove duplicate rows
df['Age_Category'] = df['Years'].apply(lambda x: 'Young' if x < 30 else 'Old') #applying func
pivot_table = df.pivot_table(index='City', values='Years', aggfunc='mean')
# Creating a Pivot Table
pivot = df.pivot_table(values='Salary', index='Department', aggfunc='mean')

#Creating a DataFrame with Date Column
df = pd.DataFrame({
    'Date': pd.date_range(start='1/1/2020', periods=10, freq='D'),
    'Value': np.random.randint(1, 100, 10)
})
# 2. Converting Date Column to Datetime Format
df['Date'] = pd.to_datetime(df['Date'])
df.set_index('Date', inplace=True). # Set Date column as index
# 3. Resampling Data by Month and Calculating Mean
monthly_avg = df.resample('M').mean()

# Merge based on common column
merged_df = pd.merge(df1, df2, on='ID', how='inner')
# Concatenation
df_concat = pd.concat([df1, df2], axis=0) # Combine along rows
grouped = df.groupby('City').agg({'Years': ['mean', 'max'], 'Salary_USD': 'sum'}) #grouping
filtered_df = df.query('Years > 30 & City == "Berlin"') #filtering
df_sorted = df.sort_values(by=['Years', 'Salary_USD'], ascending=[False, True]) #sorting
df.to_csv("output.csv", index=False) # Save as CSV
df.to_excel("output.xlsx", index=False) # Save as Excel

```

```

df_read = pd.read_csv("output.csv") ## Reading File
df.to_csv("output.csv", index=False) ## writting file
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'London', 'Berlin']
}
df = pd.DataFrame(data)
display(df)
print(df[['Name', 'Age']])
print(df[df['Age'] > 30])
df_sorted = df.sort_values(by='Age', ascending=False)

grouped = df.groupby('City')['Age'].mean()

df_missing = df.copy()
df_missing.loc[1, 'Age'] = np.nan
print(df_missing.fillna(0)) # Replace missing values with 0

merged_df = pd.merge(df1, df2, on='ID') #merging
df.info() # Summary of the DataFrame
df.describe() # Statistical summary of numerical columns
df.shape # Returns (rows, columns)
df.columns # Returns column names
df.dtypes # Data types of each column
df.rename(columns={'Name': 'Full Name', 'Age': 'Years'}, inplace=True) #renaming
df['Years'] = df['Years'].astype(float) #changing data type
df['Salary_USD'] = df['Years'] * 1.2 # Assuming conversion from another currency
df.drop(columns=['Salary_USD'], inplace=True) # Drop a column
df.drop(index=[1], inplace=True) # Drop a row by index
df.fillna(0, inplace=True) # Replace NaN with 0
df.replace('London', 'UK', inplace=True) # Replace values in a column

```